

# Find Peak Element

## 1. Problem Statement

Given an array of integers `arr`, find a peak element.  
A peak element is an element that is greater than its neighbors.

**Note:** The array may contain multiple peak elements. Return any one of them.

### Example:

Input: `arr = [1, 2, 3, 1]`

Output: 2

Explanation: `arr[2] = 3` is a peak element.

## 2. Idea

- A peak element is always greater than at least one neighbor.
- If the middle element is smaller than the right neighbor, then a peak must exist on the right side.
- Otherwise, a peak must exist on the left side.
- Use Binary Search to find any one peak element in  $O(\log n)$  time.

## 3. Algorithm

- Initialize: `s = 0, e = n-1`
- While (`s <= e`)
  - `mid = s + (e - s) / 2`
  - If (`arr[mid] < arr[mid + 1]`)  
`s = mid + 1` // go right
  - Else  
`e = mid` // go left
- Return `s`  
(or `e`, both are the peak index)

### Visualization



If `mid` is smaller than right neighbor, go right because a peak exists there. Otherwise, go left.

## 4. Dry Run

`arr = [1, 2, 3, 1]`

| s | e | mid | arr[mid] | arr[mid+1] | Condition             | Action                          |
|---|---|-----|----------|------------|-----------------------|---------------------------------|
| 0 | 3 | 1   | 2        | 3          | $2 < 3$               | $s = mid + 1 \rightarrow s = 2$ |
| 2 | 3 | 2   | 3        | 1          | $3 < 1$ is False      | $e = mid \rightarrow e = 2$     |
| 2 | 2 | 2   | 3        | 1          | Exit loop ( $s > e$ ) | Return $s = 2$                  |

Peak Element Index = 2, Value = `arr[2] = 3`

## 5. Code (Java)

```
class Solution {
    public int findPeakElement(int[] arr) {
        int s = 0, e = arr.length - 1;
        while (s <= e) {
            int mid = s + (e - s) / 2;
            if (arr[mid] < arr[mid + 1]) {
                s = mid + 1; // go right
            } else {
                e = mid; // go left
            }
        }
        return s; // or return e; both are correct
    }
}
```

### Driver Code (Example)

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 1};
        Solution obj = new Solution();
        int peakIndex = obj.findPeakElement(arr);
        System.out.println("Peak Element Index: " + peakIndex);
        System.out.println("Peak Element Value: " + arr[peakIndex]);
    }
}
```

## 6. Complexity

- Time Complexity:  $O(\log n)$   
(Binary Search)
- Space Complexity:  $O(1)$   
(Constant Space)

## 7. Key Points

- A peak element is greater than its neighbors.
- Compare `mid` with `mid + 1` only.
- If `arr[mid] < arr[mid + 1]`, go right.
- Else, go left (peak exists on left side).
- Return any one peak element index.
- Works in  $O(\log n)$  time and  $O(1)$  space.